Distributed Systems

Partitioning · Consistent Hashing · Linearizability · Two-Phase Commit
Consensus · Raft (Election & Log Replication) · ZooKeeper

Revision notes based on *Designing Data-Intensive Applications* (Kleppmann), Ch. 6 & 9,
and the Raft paper (Ongaro & Ousterhout, 2014)

Contents

1. 01 · Partitioning (Sharding) — Revision Notes
2. 02 · Consistent Hashing — Revision Notes
3. 03 · Linearizability (and CAP) — Revision Notes
4. 04 · Two-Phase Commit (2PC) and Distributed Transactions — Revision Notes
5. 05 · Distributed Consensus — Revision Notes
6. 06 · Raft — Leader Election — Revision Notes
7. 07 · Raft — Log Replication (How Writes Work) — Revision Notes
8. 08 · ZooKeeper (and Coordination Services) — Revision Notes
9. Distributed Systems — Folder Guide & Summary of Everything

01 · Partitioning (Sharding) — Revision Notes

> Source: *Designing Data-Intensive Applications* (Martin Kleppmann), **Chapter 6: Partitioning** (pp. 199–217).
> Folder guide: README.md · Next: 02-Consistent-Hashing.md

1. What partitioning is and why

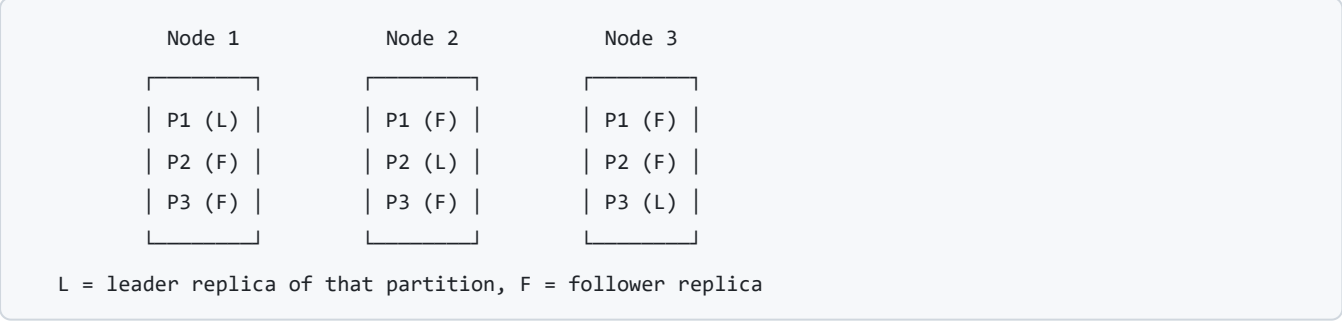
> **Partitioning = splitting a large dataset into smaller pieces (partitions) so that each piece can live on a different machine.**

Database	Calls a partition a…
MongoDB, Elasticsearch, SolrCloud	**shard**
HBase	**region**
Bigtable	**tablet**
Cassandra, Riak	**vnode**
Couchbase	**vBucket**

- **Goal: scalability.** Spread the **data** over many disks and the **query load** over many CPUs. In theory, 10 nodes handle 10× the data and 10× the throughput.
- Each record belongs to **exactly one partition**.
- ⚠️ This has **nothing to do with network partitions** (network faults), which is the "P" in CAP.

Partitioning + replication

Partitions are usually **replicated** too: each partition has copies on several nodes.



Each node is the **leader for some partitions** and a **follower for others**. Everything from the replication notes (../REPLICATION) applies per partition.

2. The core problem: skew and hot spots

- **Skew:** some partitions get **more data or more queries** than others.
- **Hot spot:** a partition with **disproportionately high load**. In the worst case, one node does all the work while 9 sit idle.
- Assigning records **randomly** spreads them evenly, but then you **don't know where a record is**, so every read must ask **every node**. ❌

3. Partitioning by key range

Give each partition a **continuous range of keys**, like volumes of a paper encyclopedia (A–B, C–D, ..., T–Z).

```
Partition 1: [A ... Bz]   Partition 2: [C ... Ez]   ...   Partition 12: [T ... Z]
```

- Ranges are **not evenly spaced**, because the data isn't. Boundaries must **adapt to the data** (set by an admin or automatically).
- Inside each partition, keys are **kept sorted** (SSTables), so **range scans are easy**.
- **Used by:** Bigtable, HBase, RethinkDB, MongoDB (before 2.4).

✅ Pros	❌ Cons
Efficient range queries (e.g. all sensor readings for one month)	**Hot spots** with certain access patterns

Hot spot example: the key is a **timestamp**, so all of today's writes go to **today's partition** while the others sit idle. **Fix: prefix the key with the sensor name**, e.g. `sensor42:2024-05-01T10:00`. Writes now spread across sensors, but reading many sensors over a time range needs **one range query per sensor**.

4. Partitioning by hash of key

Run every key through a **hash function** and give each partition a **range of hash values**.

```
hash("user_17")  = 0x3A1F...  → partition 1 (hash range 0x0000 – 0x3FFF)
hash("user_18")  = 0xC072...  → partition 4 (hash range 0xC000 – 0xFFFF)
   similar keys → completely different partitions (evenly spread)
```

- A good hash turns **skewed keys into a uniform distribution**. It doesn't need to be cryptographically strong: **MD5** (Cassandra, MongoDB), **Fowler–Noll–Vo** (Voldemort).
- ⚠️ Don't use a language's built-in hash such as Java's `Object.hashCode()` or Ruby's `Object#hash`. The **same key can hash differently in different processes**.
- Boundaries can be **evenly spaced** or **chosen pseudorandomly**. The pseudorandom version is sometimes called **consistent hashing** (see 02).

✅ Pros	❌ Cons
Even distribution, few hot spots	**Range queries become inefficient**: adjacent keys end up scattered

- MongoDB (hash sharding) sends range queries to **all partitions**. Riak, Couchbase and Voldemort **don't support** primary-key range queries at all.

The compromise: compound keys (Cassandra)

- Declare a **compound primary key** `(user_id, update_timestamp)`.
- **Only the first part is hashed** to pick the partition. The other columns are a **sorted concatenated index** inside it.
- So you can **fetch one user's updates in a time range**, sorted, from **one partition**. That's an elegant **one-to-many** model.

Key range vs hash partitioning

	Key range	Hash of key
Distribution	Can be skewed	Even
Range queries	✅ Efficient	❌ Scatter to all partitions
Hot spot risk	High (sequential keys like timestamps)	Low (except for a single hot key)
Rebalancing	Usually **dynamic splitting**	Usually a **fixed number of partitions**
Examples	HBase, Bigtable, RethinkDB	Cassandra, Riak, Voldemort, MongoDB (hashed)

5. Skewed workloads and relieving hot spots

Hashing **can't fix a single hot key**: identical keys hash identically.

- Example: a **celebrity** with millions of followers posts, causing a storm of writes to **one key** (their user ID or the post ID).
- Most databases **can't fix this automatically**, so the **application** must.

Technique: split the hot key

```
"celebrity_42"  →  "celebrity_42_00", "celebrity_42_01", ..., "celebrity_42_99"
writes: pick a random 2-digit suffix → spread over 100 keys → many partitions
reads:  must read all 100 keys and combine ❌ extra work
```

- Only do this for the **few hot keys**, which means you must **track which keys are split**.

6. Partitioning and secondary indexes

A secondary index searches **by value** ("all red cars", "all actions by user 123") and **doesn't map neatly to partitions**. There are two approaches:

(a) Document-partitioned index = local index

Each partition indexes **only its own documents**.

```
Partition 0 (cars 0–499)                Partition 1 (cars 500–999)
  color:red   → [191, 306]                color:red   → [768]
  color:black → [235]                     color:silver→ [515, 893]
  make:Honda  → [235, 306]                make:Honda  → [515]
```

- **Writes:** touch **only one partition**. ✅
- **Reads:** "find red cars" must ask **every partition** and merge the results. This is **scatter/gather**: expensive, and it suffers from **tail latency amplification**. ❌
- **Used by:** MongoDB, Riak, Cassandra, Elasticsearch, SolrCloud, VoltDB.

(b) Term-partitioned index = global index

One **global** index covering all partitions, **itself partitioned by the term** (e.g. colors a–r on partition 0, s–z on partition 1).

```
Index partition 0 (colors a–r):  color:black → [235]   color:red → [191, 306, 768]
Index partition 1 (colors s–z):  color:silver → [515, 893]   color:yellow → [...]
```

- **Reads:** go to **only the partition holding that term**. ✅
- **Writes:** one document may update **several index partitions** on different nodes. Slower and more complex. ❌
- Keeping it perfectly in sync needs a **distributed transaction**, so updates are usually **asynchronous** (e.g. **DynamoDB global secondary indexes** update "within a fraction of a second" normally).
- **Used by:** DynamoDB GSIs, Riak search, Oracle data warehouse.

	Local (document-partitioned)	**Global (term-partitioned)**
Write cost	1 partition ✅	Several partitions ❌
Read cost	**Scatter/gather to all** ❌	1 partition ✅
Freshness	Synchronous	Often **asynchronous** (may lag)

7. Rebalancing partitions

Rebalancing = moving load from one node to another when:

- query throughput grows (more CPUs),
- data grows (more disks and RAM),
- or a machine fails (others take over).

Requirements:

1. Afterwards, load is **shared fairly**.
2. The database keeps **accepting reads and writes during** rebalancing.
3. **Move no more data than necessary**.

❌ How NOT to do it: `hash(key) mod N`

```
hash(key) = 123456
10 nodes: 123456 mod 10 = 6
11 nodes: 123456 mod 11 = 3   ← moved!
12 nodes: 123456 mod 12 = 0   ← moved again!
```

When N changes, **almost every key moves**, which makes rebalancing hugely expensive.

Strategy 1: Fixed number of partitions

Create **many more partitions than nodes** at the start (e.g. **1,000 partitions for 10 nodes**, about 100 each).

```
Before (4 nodes):  N0:[p0 p4 p8 p12]  N1:[p1 p5 p9 p13]  N2:[p2 p6 p10 p14]  N3:[p3 p7 p11 p15]
Add node N4: it steals a few partitions from each existing node
After  (5 nodes):  N0:[p0 p8 p12]  N1:[p1 p5 p13]  N2:[p2 p6 p14]  N3:[p3 p7 p15]  N4:[p4 p9 p10 p11]
```

- **Only whole partitions move.** The key → partition mapping **never changes**, only partition → node.
- Old assignments serve requests while the transfer is in progress.
- More powerful nodes can take **more partitions**.
- **Used by:** Riak, Elasticsearch, Couchbase, Voldemort.
- ⚠️ The partition count is **fixed at setup**, which caps the **maximum number of nodes**. If it's too high there's overhead. If it's too low, partitions become **huge**. It's hard to pick when the data size varies a lot.

Strategy 2: Dynamic partitioning

- When a partition grows past a size limit (**HBase default: 10 GB**), it **splits in two**. When it shrinks, it **merges** with a neighbour, much like a B-tree.
- ✅ The number of partitions **adapts to the data volume**.
- ⚠️ An empty database starts with **one partition**, so one node takes all the writes at first. The fix is **pre-splitting** (HBase, MongoDB), which requires knowing the key distribution.
- **Used by:** HBase, RethinkDB (key range), MongoDB 2.4+ (key range or hash).

Strategy 3: Partitioning proportionally to nodes

- A **fixed number of partitions per node** (**Cassandra: 256 per node** by default at the time of writing).
- A new node **randomly picks existing partitions to split** and takes half of each.
- Partition size stays fairly stable as data and nodes grow together.
- Requires **hash partitioning**. It's the closest match to the original **consistent hashing** (see 02).
- **Used by:** Cassandra, Ketama.

Strategy	Number of partitions	Partition size	Examples
Fixed number	Fixed	Grows with the data	Riak, Elasticsearch, Couchbase, Voldemort
Dynamic	Grows with the data	Bounded (split and merge)	HBase, RethinkDB, MongoDB
Per node	Grows with the nodes	Fairly stable	Cassandra, Ketama

Automatic or manual rebalancing?

- **Fully automatic** is convenient but **unpredictable**: moving data is expensive and can overload the network.
- ⚠️ Combined with **automatic failure detection** it's **dangerous**. An overloaded, slow node is wrongly declared dead, rebalancing piles **more load** onto it, and you get a **cascading failure**.
- ✅ A **human in the loop** is safer. **Couchbase, Riak and Voldemort** suggest an assignment, and an **admin commits it**.

8. Request routing (service discovery)

"I want key `foo`: which IP and port do I connect to?" Partitions move over time, so somebody must track the assignment.

```
(1) Any node              (2) Routing tier              (3) Partition-aware client
client → any node         client → router → right node   client → right node directly
  (forwards if needed)      (partition-aware LB)
```

Approach	Who decides	Examples
1. Contact any node	That node forwards the request to the owner	**Cassandra, Riak** (using a **gossip protocol** to share cluster state)
2. Routing tier	A partition-aware load balancer	**MongoDB** (`mongos` + config servers), Espresso (Helix), Couchbase (`moxi`)
3. Client knows	The client connects straight to the owner	Partition-aware clients

Who keeps the map of partition → node? Often **ZooKeeper** (see 08):

```
nodes register in ZooKeeper ──▶ ZooKeeper holds the authoritative partition → node map
routing tier / clients subscribe ──▶ notified on every change
```

- **Uses ZooKeeper:** HBase, SolrCloud, Kafka (historically), and LinkedIn Espresso through Helix.
- **Gossip instead:** Cassandra and Riak. No external dependency, but more complexity in the nodes.
- Finding the router or node IPs themselves changes slowly, so **DNS** is usually enough.

Parallel query execution

NoSQL stores mostly do single-key reads and writes (plus scatter/gather). **MPP** (massively parallel processing) analytic databases split complex queries with joins and aggregations into **stages that run in parallel** across nodes.

9. Conclusion

Partitioning (sharding) splits data that's too big for one machine into partitions spread over many nodes, so that data and queries **scale out**. Each partition is usually replicated too. The enemy is **skew and hot spots**. **Key-range partitioning** keeps keys sorted, which makes **range scans** cheap, but sequential keys like timestamps create hot spots. **Hash partitioning** spreads keys evenly but **loses range queries**. Cassandra's **compound keys** combine both, and a single **hot key** must be split by the application. **Secondary indexes** are either **local** (cheap writes, scatter/gather reads) or **global** (cheap reads, expensive and often async writes). **Rebalancing** must stay online and move little data: never `hash mod N`. Instead use a **fixed number of partitions**, **dynamic split and merge**, or **partitions per node**, ideally with a **human approving** moves. **Request routing** (contact any node, a routing tier, or a smart client) relies on **ZooKeeper** or **gossip** to know which node owns which partition.

Interview one-liners

- **Partitioning:** "Split data across nodes so each holds a subset. It scales storage and throughput, and is usually combined with replication."
- **Range vs hash:** "Range keeps order for scans but risks hot spots. Hash spreads load evenly but kills range queries."
- **Local vs global index:** "Local means cheap writes and scatter/gather reads. Global means single-partition reads and multi-partition, usually async, writes."

- **Rebalancing:** "Never use hash mod N, which moves almost every key. Use many fixed partitions per node and move whole partitions."
- **Routing:** "A coordination service like ZooKeeper, or gossip, keeps the partition-to-node map, and requests are routed by any node, a routing tier or a smart client."

Self-check questions

1. Why partition? How does it relate to replication?
2. What are skew and hot spots? Why is random assignment a bad idea?
3. Compare key-range and hash partitioning. Give a hot-spot example for key range and its fix.
4. How do Cassandra's compound primary keys give you both even distribution and range scans?
5. How do you handle a single extremely hot key?
6. Compare document-partitioned (local) and term-partitioned (global) secondary indexes.
7. Why is `hash mod N` bad for rebalancing? Use the 123456 example.
8. Compare fixed-number, dynamic, and per-node partitioning.
9. Why can fully automatic rebalancing cause cascading failures?
10. Describe the three request-routing approaches and how ZooKeeper or gossip helps.

02 · Consistent Hashing — Revision Notes

> Source: *Designing Data-Intensive Applications*, Chapter 6, the "Consistent Hashing" sidebar (p. 204) and "Partitioning proportionally to nodes" (pp. 212–213). ⚠️ The book covers this only briefly and even recommends avoiding the term. Sections marked 📘 **Beyond the book** add the standard **hash-ring** explanation (Karger et al., 1997; Amazon Dynamo, 2007) that interviews expect. Folder guide: README.md · Previous: 01-Partitioning.md · Next: 03-Linearizability.md

1. What the book says

- **Consistent hashing** (Karger et al.) was designed to spread load across an **internet-wide system of caches**, such as a **CDN**.
- It uses **randomly chosen partition boundaries**, so it needs **no central control or consensus**.
- ⚠️ "Consistent" here has **nothing to do** with replica consistency or ACID consistency. It describes a **rebalancing approach**.
- Kleppmann's view: the original approach **doesn't work very well for databases**, and database docs often use the term **inaccurately**, so it's **better to just say "hash partitioning"**.
- The closest real database version is **Cassandra / Ketama**: a **fixed number of partitions per node**, where a new node **randomly splits existing partitions** and takes half of each (see 01 §7).

2. The problem it solves: `hash mod N`

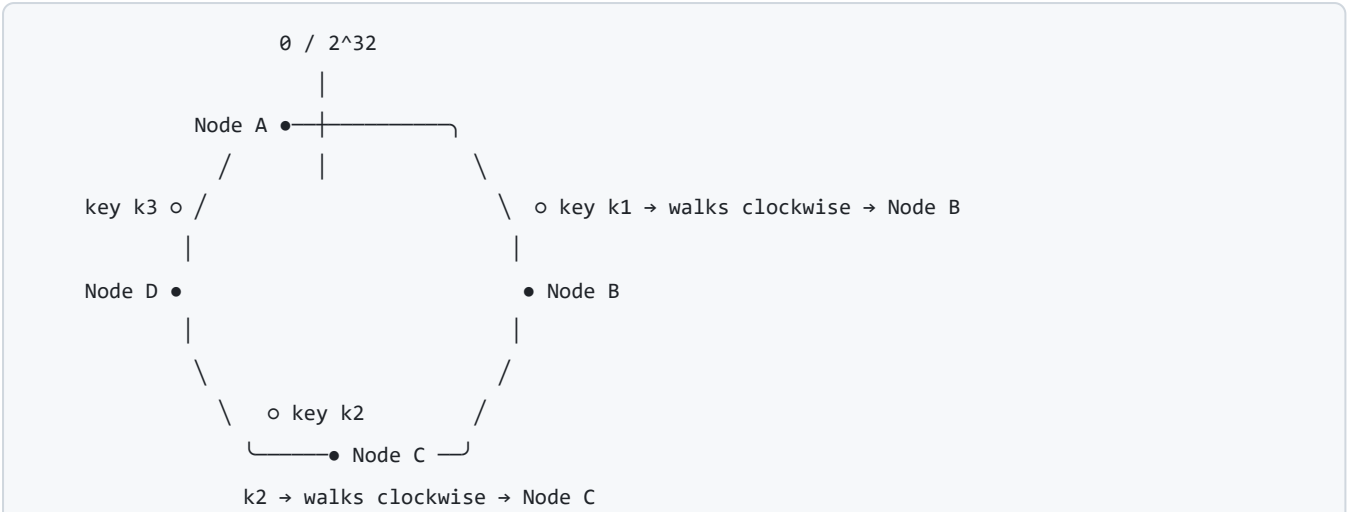
```
node = hash(key) mod N

N = 10 → 11 nodes:  a key only stays put if hash mod 10 == hash mod 11
                    → roughly 90% of ALL keys move ❌
```

Goal: when a node is **added or removed**, move only about **1/N of the keys**, just the share the new node should own.

3. The hash ring 📘 Beyond the book

1. Imagine the hash output space (e.g. **0 … $2^{32} - 1$**) bent into a **circle (ring)**.
2. Hash each **node** (e.g. its IP) onto the ring.
3. Hash each **key** onto the ring and **walk clockwise**: the **first node** you meet owns the key.



Adding a node

```
Before:  ... Node A ──────────────────────── Node B ...
             keys in (A, B] → B
Add Node X between A and B:
         ... Node A ──────── Node X ──────── Node B ...
             keys in (A, X] → X  (moved from B)
             keys in (X, B] → B  (unchanged)
```

- **Only the keys between X's predecessor and X move**, all of them taken from **one neighbour**. Everything else stays put. ✅

Removing a node

- Its keys move to the **next node clockwise**. No other keys move. ✅

4. The problem: uneven load → virtual nodes 📘 Beyond the book

With only a few nodes, random positions leave **uneven arcs**. Some nodes own far more of the ring than others, and a removed node dumps **all** its load on **one** neighbour.

> **Virtual nodes (vnodes):** each physical node is placed on the ring **many times** (e.g. A#1, A#2, ... A#100).

```
Ring: A#1  C#4  B#2  A#7  D#1  B#9  C#2  A#3  D#6  B#5 ...
      (each physical node owns many small, scattered arcs)
```

Benefit	Why
Even load	Many small arcs average out
Smooth rebalancing	A new node takes small slices from **many** nodes, and a removed node's load spreads over **many** nodes
Heterogeneous hardware	A bigger machine gets **more vnodes**, so it takes more load

This is exactly the book's **"partitions proportional to nodes"** idea: **Cassandra** used **256 tokens per node** by default when the book was written (newer versions use fewer).

5. Replication on the ring 📘 Beyond the book

In **Dynamo-style** systems, a key is stored on its owner **plus the next N − 1 distinct physical nodes clockwise**. This is the **preference list**.

```
key k → owner Node B, replicas: Node C, Node D   (n = 3)
```

That's where the **n** in quorums comes from (see REPLICATION/07-Quorums.md).

6. Where it's used

System	How
Akamai CDN (the original Karger et al. use) 📘	Spread cached content over cache servers
Amazon Dynamo 📘	Ring + vnodes + preference lists
Cassandra, Riak	Token ring with vnodes (Cassandra: "partitions per node")
Ketama (memcached clients)	Consistent hashing for cache servers
Load balancers and caches 📘	Sticky routing that survives servers being added or removed

7. Alternatives 📘 Beyond the book

Technique	Idea
Rendezvous (HRW) hashing	For each key, compute `hash(key, node)` for every node and pick the highest. Minimal movement, no ring
Jump consistent hash (Google)	Tiny, fast, no memory, but nodes must be numbered 0…N−1 (no arbitrary removal)
Fixed partitions + map (the book's preferred way)	Many partitions, and a lookup table (often in ZooKeeper) maps partitions to nodes

The book notes that **newer hash functions** achieve the same effect with **less metadata**.

8. Comparison

	`hash mod N`	**Consistent hashing (ring + vnodes)**	**Fixed partitions + map**
Keys moved when adding 1 node to N	**~all** ❌	**~1/(N+1)** ✅	**~1/(N+1)** ✅ (whole partitions)
Central metadata needed	No	No (positions come from hashes)	Yes (partition → node map)
Load evenness	Even	Even **with vnodes**	Even (assign partitions)
Range queries	❌	❌	❌ (hash-based)

9. Conclusion

Consistent hashing places both **nodes and keys on a hash ring**, and each key belongs to the **first node clockwise**. Adding or removing a node only moves the keys in **one arc** (about 1/N of the data), whereas `hash mod N` reshuffles almost everything. Plain rings give **uneven arcs**, so real systems use **virtual nodes**: many positions per physical node for even load, smooth rebalancing and support for bigger machines. Dynamo-style stores replicate each key to the **next N − 1 nodes** clockwise. The book's caution is that "consistent" has nothing to do with consistency, the original idea fits CDNs better than databases, and in databases it shows up as **Cassandra's partitions-per-node** scheme, so "hash partitioning" is the clearer term.

Interview one-liners

- **Consistent hashing:** "Hash nodes and keys onto a ring, and a key goes to the next node clockwise. Adding or removing a node only remaps the neighbouring arc, about 1/N of the keys."
- **Virtual nodes:** "Each server takes many ring positions, which evens out load, spreads rebalancing across many nodes and supports heterogeneous capacity."
- **Why not mod N:** "Changing N remaps nearly every key, which is a massive data movement."

Self-check questions

1. Why is `hash(key) mod N` bad when N changes?
2. Draw a hash ring and show which node owns a key.
3. What moves when a node is added? When a node is removed?
4. Why do we need virtual nodes? List three benefits.
5. How do Dynamo-style systems pick replicas on the ring?
6. Why does Kleppmann suggest avoiding the term "consistent hashing"?
7. How does Cassandra's "partitions proportional to nodes" relate to consistent hashing?
8. Name two alternatives to ring-based consistent hashing.

03 · Linearizability (and CAP) — Revision Notes

> Source: *Designing Data-Intensive Applications*, **Chapter 9**, "Consistency Guarantees" and "Linearizability" (pp. 321–338). Folder guide: README.md · Previous: 02-Consistent-Hashing.md · Next: 04-Two-Phase-Commit.md

1. Why we need something stronger than eventual consistency

- Most replicated databases offer **eventual consistency**, which would be better named **convergence**: stop writing, wait, and replicas eventually agree.
- It's **very weak**: until then, reads can return **anything**, even an old value right after you wrote a new one.
- It's hard for developers because a database **doesn't behave like a variable** in a normal program.
- **Stronger models are easier to use correctly, but cost performance and fault tolerance.**

⚠️ **Distributed consistency ≠ transaction isolation.**

- Isolation is about **races between concurrent transactions**.
- Consistency models are about **coordinating replicas despite delays and faults**.

2. What linearizability is

> **Linearizability:** make the system **look as if there is only one copy of the data**, and **every operation on it is atomic**.

- Other names: **atomic consistency, strong consistency, immediate consistency, external consistency**.
- **It's a recency guarantee:** once a write completes, **every** later read, from **any** client, sees that value or a newer one. **No stale replica or cache values.**

The Alice and Bob example (not linearizable)

```
World Cup final ends.
Alice refreshes → replica 1 (up to date) → "Germany won!" → she tells Bob
Bob refreshes AFTER hearing Alice → replica 2 (lagging) → "game still on" ❌
```

Bob's request **started after Alice's finished**, so it must not return **older** data.

3. What makes a system linearizable

Think of a **register** x: one key, row or document. Each operation takes effect **atomically at some instant between its request and its response**.

```
time ──────────────────────────────────────────────────────────────▶
Client C:        |──────── write(x, 1) ────────|
Client A: |read→0|     |read→0 or 1|      |read→ MUST be 1|
Client B:                  |read→1|  |read→ MUST be 1 (A or B already saw 1)|
```

Rules:

1. A read that **finishes before** the write starts returns the **old** value.
2. A read that **starts after** the write finishes returns the **new** value.
3. A read **concurrent** with the write may return either, **but**:
4. ⭐ **Once any read has returned the new value, every later read (on any client) must also return the new value.** The value **flips atomically** at one point and **never flips back**.

- Joining each operation's "effective instant" must give a valid **sequential order** that always **moves forward in time**.
- **compare-and-set** `cas(x, old, new)` succeeds only if x still equals `old`. This is how linearizable systems build locks.
- You can **test** linearizability (expensively) by recording request and response times and checking whether a valid order exists.

Linearizability vs serializability ⚠️

	Serializability	**Linearizability**
About	**Transactions** (multi-object) — isolation	**Single object** (register) — recency
Guarantee	The result equals **some** serial order (which may differ from real time)	Operations appear in **real-time order**, and reads see the **latest** value
Prevents write skew?	✅	❌ (no grouping of operations)

- **Both together = strict serializability** (strong one-copy serializability).
- **2PL and actual serial execution** are typically linearizable.
- **SSI is NOT linearizable**: it reads from a snapshot, so it deliberately ignores newer writes.

4. When you need linearizability

Use case	**Why**
Locking and leader election	Every node must **agree who holds the lock** or who is leader, otherwise you get **split brain**. **ZooKeeper / etcd** provide this with consensus. Oracle RAC uses linearizable per-page locks
Uniqueness constraints	Usernames, file paths, **no overselling stock**, **no double-booked seats**, **balance ≥ 0**. All need **one up-to-date value** everyone agrees on (like a lock or CAS on the username)
Cross-channel timing dependencies	Two communication paths can race (see below)

> Note: ZooKeeper and etcd **writes** are linearizable, but **reads may be stale** by default. Use `sync()` (ZooKeeper) or a **quorum read** (etcd) for linearizable reads.

Cross-channel race (image resizer)

```
1. web server writes full-size photo → file storage (replicating...)
2. web server sends "resize photo 123" → message queue  (FAST)
3. resizer reads the queue → fetches photo 123 from file storage
   → storage replica not updated yet → gets OLD or NO image ❌
```

There are **two channels** (storage and queue). Without linearizable storage, the queue can **overtake** replication.

5. Implementing linearizable systems

Replication method	Linearizable?	Why
Single-leader	⚠️ **Potentially**	If you read from the leader or synchronous followers. But a node may **wrongly think it's the leader**, and async failover can **lose committed writes**
Consensus algorithms (Raft, Zab)	✅ **Yes**	Single-leader-like, with protection against **split brain and stale replicas**. **ZooKeeper and etcd** work this way
Multi-leader	❌ No	Concurrent writes on several nodes, replicated asynchronously, so **conflicts**
Leaderless (Dynamo-style)	❌ **Probably not**	**LWW with clocks** breaks it, **sloppy quorums** break it, and even **strict quorums** can fail (below)

Why strict quorums still aren't linearizable

```
n = 3, w = 3, r = 2. Writer sets x = 1 (arrives at replicas at different times)
Client A reads replicas {1, 2} → sees 1 on replica 1 → returns 1
Client B (starts AFTER A finished) reads replicas {2, 3} → both still 0 → returns 0 ❌
```

- It can be fixed with **synchronous read repair** plus **reading before writing**, at a **performance cost**.
- Even then, **compare-and-set** needs **consensus**.
- **Assume Dynamo-style systems are not linearizable.**

6. The cost of linearizability and the CAP theorem

Two datacenters, and the link between them breaks:

Setup	What happens during the network cut
Multi-leader	Each datacenter keeps working, and writes queue up and sync later. ✅ **Available**, ❌ not linearizable
Single-leader	Clients in the **follower** datacenter can't reach the leader, so **no writes and no linearizable reads**. ❌ **Unavailable** there

CAP, stated properly

- **Network partitions will happen.** They're a fault, **not a choice**.
- So **CAP really means: when partitioned, be either Consistent (linearizable) or Available.**
  - **Need linearizability:** disconnected replicas must **wait or return errors**, so they're **unavailable**.
  - **Don't need linearizability:** each replica can serve requests independently, so it's **available but not linearizable**.
- "Pick 2 of 3" is **misleading**, because you can't opt out of partitions.
- CAP was useful as a **rule of thumb** that sparked the **NoSQL** movement. But formally it covers **only linearizability and only network partitions**, not delays, dead nodes or other trade-offs, so it has **little practical value** today. **Avoid CP/AP labels**.

> 📘 *Beyond the book:* **PACELC** extends CAP: during a **P**artition choose **A** or **C**, **E**lse (normal operation) choose **L**atency or **C**onsistency. That captures the point below.

Linearizability is slow, all the time

- Even **multi-core CPU RAM isn't linearizable** (each core has caches and store buffers) unless you use memory barriers. The reason is **performance, not fault tolerance**.
- Many databases drop linearizability **for speed**.
- **Attiya & Welch:** linearizable response time is **at least proportional to the uncertainty of network delays**. **No faster algorithm exists.** Weaker models can be much faster.

7. Conclusion

Linearizability makes a replicated system behave like **one copy of the data with atomic operations**. Once a write completes, or any read has seen the new value, **every later read everywhere** sees it. It's a **single-object recency guarantee**, distinct from **serializability** (multi-object transaction isolation), and combining them gives **strict serializability**. You need it for **locks and leader election**, **uniqueness constraints** (usernames, seats, stock), and avoiding **cross-channel races**. **Consensus-based** systems like ZooKeeper and etcd provide it. Single-leader can, if you really know who the leader is. Multi-leader and Dynamo-style quorums **don't**. The price is the **CAP trade-off**: during a network partition a linearizable system must make some replicas **unavailable**. More importantly, linearizability is **slow all the time**, since response time is bounded below by network delay uncertainty. That's why many systems choose weaker models.

Interview one-liners

- **Linearizability:** "The system behaves as if there's a single copy and operations are atomic. After a write completes, every read sees it, so it's a recency guarantee."
- **vs serializability:** "Serializability is transaction isolation in some serial order. Linearizability is real-time recency on single objects. Both together is strict serializability."
- **CAP:** "Partitions are inevitable, so during one you choose linearizability or availability. The bigger everyday cost is latency."
- **Quorums:** "Even w + r > n isn't linearizable with variable network delays."

Self-check questions

1. Why is eventual consistency hard for developers?
2. Define linearizability using the Alice and Bob example.
3. What does "once any read returns the new value, all later reads must" mean?
4. Linearizability vs serializability: what's strict serializability? Is SSI linearizable?
5. Give three use cases that require linearizability.
6. Explain the image-resizer cross-channel race.
7. Which replication methods can be linearizable, and why?
8. Show why a strict quorum read can still be non-linearizable.
9. State CAP correctly. Why is "pick 2 of 3" misleading?
10. Why is linearizability slow even without network faults?

04 · Two-Phase Commit (2PC) and Distributed Transactions — Revision Notes

> Source: *Designing Data-Intensive Applications*, **Chapter 9**, "Atomic Commit and Two-Phase Commit (2PC)" and "Distributed Transactions in Practice" (pp. 354–364). Folder guide: README.md · Previous: 03-Linearizability.md · Next: 05-Distributed-Consensus.md

1. The atomic commit problem

Atomicity (the A in ACID) means a transaction **either commits fully or aborts fully**, with no half-finished state.

On a single node

1. Write the transaction's data to disk (the WAL).
2. Append a **commit record**.

The moment the **disk finishes writing the commit record** is the deciding moment. **One disk controller** makes the commit atomic.

Across several nodes (partitions, or a global secondary index on another node)

Just sending "commit" to every node **isn't safe**:

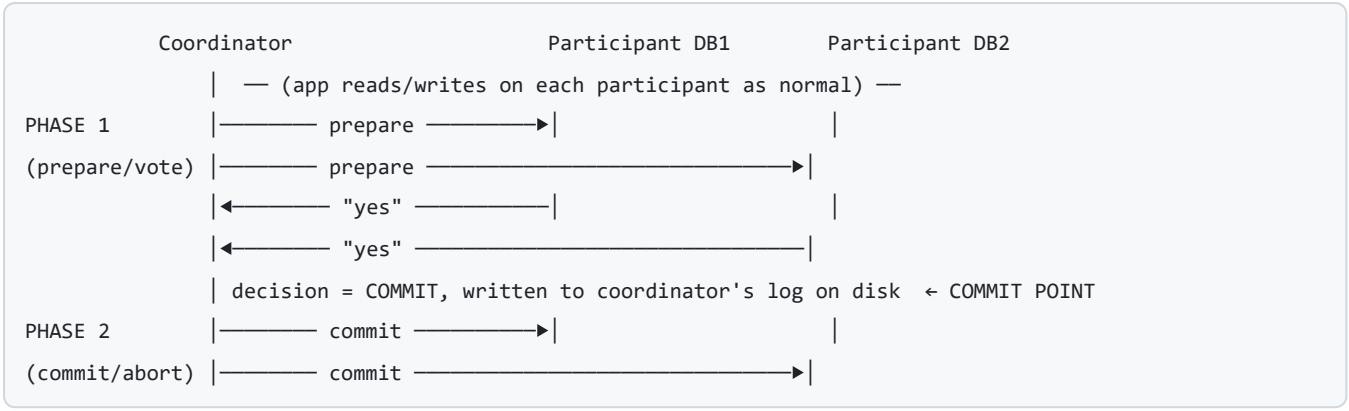
- one node finds a **constraint violation** and must abort, while others commit
- a commit request is **lost in the network**, so that node times out and aborts
- a node **crashes** before writing its commit record and rolls back

➡️ Nodes end up **inconsistent**. **A commit must be irrevocable**: once data is committed it's visible and others rely on it (read committed). So **a node may only commit once it's sure everyone will commit.**

2. Two-phase commit (2PC)

> ⚠️ **2PC ≠ 2PL.** Two-phase **commit** gives **atomic commit across nodes**. Two-phase **locking** gives **serializable isolation**. They're unrelated.

- A new component: the **coordinator** (transaction manager), often a **library inside the application** (e.g. in a Java EE container), or a separate service. Examples: Narayana, JOTM, BTM, MSDTC.
- The database nodes are **participants**.



- **All participants say "yes"** → phase 2 sends **commit**.
- **Any participant says "no"** (or a prepare times out) → phase 2 sends **abort** to all.

> 💍 **Marriage analogy:** the minister asks each partner "Do you?" (prepare). Both say "I do" (yes votes). The minister pronounces them married (commit). If either says no, the ceremony is aborted.

3. Why it works: a system of promises

1. The app gets a **globally unique transaction ID** from the coordinator.
2. It runs a normal **single-node transaction on each participant**, tagged with that ID. Anyone can abort at this stage.
3. When ready, the coordinator sends **prepare** (with the ID) to all. If any fails or times out → **abort** all.
4. A participant that replies **"yes"** must first make sure it **can definitely commit under all circumstances**: data written to disk, constraints checked. ⭐ **It gives up the right to abort**, without committing yet.
5. The coordinator collects all votes and **decides**, writing the decision to its **log on disk**. This is the **commit point**.
6. It sends commit or abort to everyone, **retrying forever** until each one succeeds. A crashed participant commits when it recovers, because it **promised**.

⭐ **Two points of no return:**

- a participant voting **"yes"**
- the coordinator **deciding**

(On a single node, both happen in one step: writing the commit record.)

4. Coordinator failure: the weak spot

- The coordinator crashes **before** sending prepare → participants can **safely abort**.
- The coordinator crashes **after** participants voted "yes" → participants **can't abort or commit on their own**. They must **wait**.

> A participant that has voted "yes" but hasn't heard the outcome is **in doubt** (uncertain).

```
Coordinator decided COMMIT → sent commit to DB2 ✅ → CRASH 💥 before reaching DB1
DB1: voted yes, no decision received → IN DOUBT
   • abort on timeout?   ❌ DB2 already committed → inconsistent
   • commit on its own?  ❌ another participant might have voted no
   → must WAIT for the coordinator to recover
```

- The only way out is for the **coordinator to recover** and read its **log**. Any transaction without a commit record in the log → **abort**.
- So 2PC's commit point is really **a single-node atomic commit on the coordinator**.
- 2PC is a **blocking** atomic commit protocol.

Three-phase commit (3PC)

- It's **non-blocking** in theory, but assumes **bounded network delay and bounded response times**.
- Real networks have unbounded delays, so 3PC **can't guarantee atomicity** there.
- Non-blocking atomic commit needs a **perfect failure detector**, which doesn't exist (a timeout can't tell a crash from a slow network).
- ➡️ **2PC is still used** despite the blocking problem.

5. Distributed transactions in practice

They have a **mixed reputation**: an important safety guarantee, but **operational pain** and **slowness**. For example, **MySQL distributed transactions** are reported to be **over 10× slower** than single-node ones, because of extra **fsyncs** and **network round trips**. Many cloud services don't offer them.

Type	Meaning	Works well?
Database-internal	All participants run the **same database software** (VoltDB, MySQL Cluster NDB)	✅ Often well, since they can use custom optimized protocols
Heterogeneous	**Different technologies** (two databases from different vendors, or a database plus a message broker)	⚠️ Much harder

Exactly-once message processing (the heterogeneous win)

- Atomically commit **"message acknowledged" + "database writes"** together.
- If either fails, **both abort**, and the broker can **safely redeliver**. The message is effectively processed **exactly once**.
- ⚠️ This only works if **every** side effect supports the same atomic commit protocol. An email server without 2PC might **send the email twice**.

XA transactions

- **X/Open XA (1991)** is the standard for **2PC across heterogeneous systems**.
- **Supported by:**
  - **databases:** PostgreSQL, MySQL, DB2, SQL Server, Oracle
  - **message brokers:** ActiveMQ, HornetQ, MSMQ, IBM MQ
- **It's a C API, not a network protocol.** In Java it's **JTA**, used through **JDBC** and **JMS** drivers.
- The coordinator is usually a **library in the application process** with its log on the **application server's local disk**.

- If the app server dies, participants are **stuck in doubt** until it **restarts** and reads the log.
- The database **can't contact the coordinator directly**. Everything goes through its client library.

Holding locks while in doubt (why "in doubt" is so bad)

- Transactions hold **row locks** (exclusive locks for writes, plus shared locks for reads under 2PL) **until commit or abort**.
- If the coordinator takes **20 minutes** to restart, the locks are held for **20 minutes**. If its **log is lost**, they're held **forever**.
- Other transactions touching those rows **block**, so large parts of the app may become **unavailable**.

Recovering from coordinator failure

- **Orphaned in-doubt transactions** do happen (a lost or corrupted log, software bugs).
- They survive **database restarts**, because correct 2PC must keep their locks.
- The only fix is **manual**: an admin checks every participant and applies the same outcome, usually **under pressure during an outage**.
- **Heuristic decisions** are an emergency escape hatch: a participant decides on its own. "Heuristic" is a **euphemism for probably breaking atomicity**. Use it only in catastrophes.

6. Limitations of distributed transactions

Limitation	Detail
The coordinator is a database	It stores transaction outcomes. If it's **not replicated**, it's a **single point of failure**, and many coordinators **aren't highly available** by default
Stateful application servers	The coordinator's log lives on the app server, so app servers are **no longer stateless**. The log is as critical as the databases
Lowest common denominator	XA **can't detect deadlocks across systems** and **doesn't work with SSI**
Amplifies failures	**Every** participant must respond, so if **any** part is broken the transaction fails. That runs counter to fault tolerance

> 📘 *Beyond the book:* a common alternative in microservices is the **Saga pattern**: a chain of **local transactions**, each with a **compensating action** to undo it if a later step fails. It gives **eventual consistency without locks or a blocking coordinator**. The book discusses similar alternatives in Chapters 11 and 12.

7. 2PC vs consensus (preview)

	2PC	**Fault-tolerant consensus (Raft, Paxos, Zab)**
Who decides	A **fixed coordinator** (not elected)	An **elected leader**
Votes needed	**All** participants must say yes	A **majority**
Coordinator or leader dies	**Blocks** (in doubt)	**Elects a new leader** and continues
Satisfies "termination"?	❌ No	✅ Yes (if a majority is alive)

➡️ 2PC is **a kind of consensus algorithm, but not a very good one**. See 05.

8. Conclusion

Atomic commit across nodes needs every participant to agree to commit or abort, because a commit **can't be revoked** once visible. **Two-phase commit** does this with a **coordinator**. In **phase 1 (prepare)**, each participant makes sure it *can* commit and votes "yes", **giving up its right to abort**. The coordinator then **writes its decision to disk (the commit point)**, and in **phase 2** it sends commit or abort, **retrying forever**. Its fatal flaw is **coordinator failure**: participants that voted "yes" are **in doubt**, holding **locks** until the coordinator recovers (or forever, if its log is lost, needing **manual or heuristic** resolution). 3PC can't fix this on real networks. In practice, **database-internal** distributed transactions can work well, while **heterogeneous XA** transactions enable **exactly-once processing** but bring **single points of failure, stateful app servers, no cross-system deadlock detection and failure amplification**. Fault-tolerant **consensus** improves on 2PC by **electing** the leader and needing only a **majority**.

Interview one-liners

- **2PC:** "The coordinator asks every participant to prepare. If all vote yes, it logs commit and tells everyone to commit. Otherwise it aborts. A yes vote is a promise that you can commit."
- **The problem:** "If the coordinator dies after the votes, participants are in doubt and hold their locks until it recovers. 2PC is blocking."
- **XA:** "The standard C API for 2PC across heterogeneous databases and message brokers. It enables exactly-once processing but is operationally painful."
- **2PC vs consensus:** "2PC needs all participants and a fixed coordinator. Consensus needs a majority and elects a new leader on failure."

Self-check questions

1. Why can't you just send "commit" to every node in a distributed transaction?
2. Describe both phases of 2PC with a diagram.
3. What are the two "points of no return"? Why do they guarantee atomicity?
4. What does "in doubt" mean? Why can't a participant just time out?
5. How does the coordinator recover? What is the commit point?
6. Why doesn't 3PC solve the problem in practice?
7. Database-internal vs heterogeneous distributed transactions?
8. How do distributed transactions enable exactly-once message processing?
9. What is XA? Where does its coordinator usually live, and why is that a problem?
10. Why is holding locks while in doubt so damaging? What are heuristic decisions?
11. List four limitations of distributed transactions.
12. 2PC vs 2PL vs consensus: what's the difference?

05 · Distributed Consensus — Revision Notes

> Source: *Designing Data-Intensive Applications*, **Chapter 9**, "Ordering Guarantees" and "Fault-Tolerant Consensus" (pp. 339–352, 364–369). Folder guide: README.md · Previous: 04-Two-Phase-Commit.md · Next: 06-Raft-Leader-Election.md

1. What consensus is

> **Consensus = getting several nodes to agree on something**, in a way that's **final** and survives failures.

It sounds simple, but **many broken systems** were built by people who thought it was easy.

Where nodes must agree:

Problem	What they agree on
Leader election	Who the **one** leader is, to avoid **split brain**
Atomic commit	**Commit or abort** a distributed transaction (all the same way)
Uniqueness / locks	Who gets the **last seat**, the **username** or the **lock**
Total order broadcast	The **same order** of messages on every node

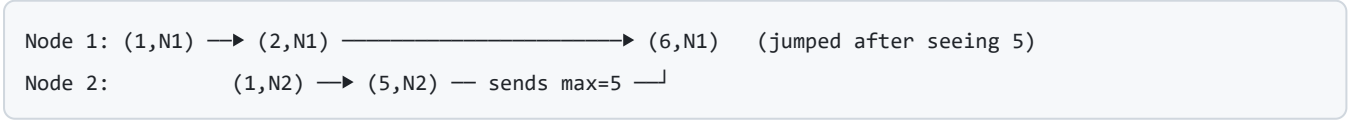
2. Background: ordering, causality and total order

Causality

- **Cause comes before effect**: a question before its answer, a row created before it's updated.
- Causality is a **partial order**. Some events are **concurrent** (neither caused the other), so history **branches and merges**, like Git.
- **Linearizability** is a **total order**: one single timeline. Linearizability **implies** causality.
- 📌 **Causal consistency** is the **strongest model that doesn't slow down with network delays and stays available during network failures**.

Lamport timestamps

- Each node keeps a counter and stamps events with **(counter, nodeID)**.
- Every node and client tracks the **maximum counter it has seen** and sends it with every request. On seeing a bigger one, it **jumps its own counter up** to match.
- This gives a **total order consistent with causality**: compare counters first, then node IDs.



⚠️ Timestamp ordering isn't enough

- **Example:** two users register the **same username** at the same time.
- Lamport timestamps tell you which came first **only after you've collected every operation**.

- A node can't decide **right now** whether someone else is claiming the name at this moment.
- ➡️ You need to know when the order is **final**. That's **total order broadcast**.

Total order broadcast (atomic broadcast)

A protocol for delivering messages to all nodes with two safety properties:

1. **Reliable delivery:** if one node gets a message, **all** get it (nothing lost).
2. **Totally ordered delivery:** every node gets messages in the **same order**.

- It's equivalent to building a **shared append-only log**.
- The order is **fixed when a message is delivered**. No message can be slipped in earlier later on.

Uses:

- **State machine replication:** same writes in the same order means identical replicas.
- **Serializable transactions** as ordered stored procedures.
- **Lock services with fencing tokens:** the log position is a monotonically increasing token, which ZooKeeper calls the **zxid**.

The equivalence (a deep result)

> **Linearizable compare-and-set (or increment-and-get) ⇔ total order broadcast ⇔ consensus.** Solve one and you can build the others.

- **Linearizable storage from total order broadcast:** to claim a username, **append** "I claim X" to the log, **wait for it to come back**, and check whether **your message is the first** for X. Writes are linearizable. Reads need extra steps: sequence the read through the log, ZooKeeper's `sync()`, or etcd's quorum read.
- **Total order broadcast from linearizable storage:** use **increment-and-get** on a linearizable counter as a **gap-free sequence number** for every message.

3. The formal definition of consensus

One or more nodes **propose** values, and the algorithm **decides** one of them. It must satisfy:

Property	Meaning	Type
Uniform agreement	**No two nodes decide differently**	Safety
Integrity	**No node decides twice** (no changing your mind)	Safety
Validity	A decided value was **proposed by some node** (rules out "always decide null")	Safety
Termination	Every node that doesn't crash **eventually decides**	**Liveness** (fault tolerance)

- Without termination it's easy: hard-code a **"dictator"** node. But if it dies, nothing can be decided. **That's exactly 2PC's problem.**
- Consensus assumes a crashed node **may never come back** ("buried under 30 feet of mud"), so any algorithm that **waits for recovery** (like 2PC) **fails termination**.

- ⭐ **Termination needs a majority of nodes working.** Safety (agreement, integrity, validity) is kept **even if a majority fails**: the system may **stop**, but it never makes a **wrong** decision.
- Most algorithms assume **no Byzantine faults** (lying nodes). Byzantine-tolerant consensus needs **fewer than 1/3 faulty nodes**.

The FLP impossibility result

- **Fischer, Lynch and Paterson** proved that **no algorithm can always reach consensus if a node may crash**.
- But that's in the **asynchronous model**: no clocks, no timeouts.
- With **timeouts** (even unreliable ones), or even just **random numbers**, consensus **becomes solvable**.
- ➡️ Consensus is possible **in practice**.

4. Consensus algorithms

The best-known ones: Viewstamped Replication (VSR), Paxos, Raft, Zab.

- They're similar but not identical.
- Most decide on a **sequence of values**, so they're really **total order broadcast** algorithms. That's more efficient than one value at a time (for Paxos, this optimization is **Multi-Paxos**).
- Total order broadcast is **repeated rounds of consensus**:
  - **agreement** → same order everywhere
  - **integrity** → no duplicates
  - **validity** → nothing fabricated
  - **termination** → nothing lost

Algorithm	Used in
Raft	**etcd** (Kubernetes), Consul 📘, CockroachDB 📘, TiKV 📘, Kafka KRaft 📘
Zab	**ZooKeeper**
Paxos / Multi-Paxos	Google Chubby 📘, Spanner 📘
Viewstamped Replication	Research and early systems

📘 = examples beyond the book.

> ⚠️ Kleppmann's advice: **don't implement consensus yourself**. It's notoriously hard. Use ZooKeeper or etcd.

5. Single-leader replication vs consensus

- Single-leader replication is **total order broadcast** (the leader orders all writes). Why didn't Chapter 5 need consensus?
- **Manually chosen leader** = a **dictatorship**. It works, but it **fails termination**: humans must step in when the leader dies.
- **Automatic failover** needs everyone to **agree on the new leader**, which needs **consensus**. But consensus algorithms use a leader…
- 🐔🥚 "*To elect a leader we first need a leader.*" How is this resolved?

6. Epoch numbers and quorums: how it's resolved

Consensus protocols **don't guarantee a unique leader at all times**. They guarantee **a unique leader within each epoch**.

Algorithm	Name for an epoch
Paxos	**Ballot number**
Viewstamped Replication	**View number**
Raft	**Term number**

- When the leader seems dead, an **election** starts with a **higher epoch number**. Epochs are **totally ordered and increasing**.
- If two leaders conflict, **the higher epoch wins**.

Two rounds of voting:

1. **Elect a leader:** a quorum votes for it in epoch e.
2. **Decide each proposal:** the leader sends the value, and a **quorum** must accept it. A node accepts **only if it hasn't seen a higher epoch**.

⭐ **The two quorums must overlap.** If a proposal gets a quorum and none of those nodes knows a higher epoch, then **no newer leader exists**, and the leader can safely decide.

```
5 nodes. Election quorum (epoch 7): {A, B, C}.  Proposal quorum: {C, D, E}.
Overlap = C. If a newer epoch 8 existed, C would know → proposal rejected.
```

> "**The truth is defined by the majority.**" A node can't trust its own belief that it's the leader. It must get **votes**.

7. Consensus vs two-phase commit

	2PC	Consensus
Leader or coordinator	**Not elected**, fixed	**Elected**, replaceable
Votes needed	**Every** participant	**Majority**
After failure	**Blocks** (in doubt)	A **recovery process** brings nodes to a consistent state under a new leader
Termination	❌	✅ (with a majority alive)

8. Limitations of consensus

Limitation	Detail
Synchronous replication cost	Voting before deciding is **synchronous replication**. Many databases prefer **async** for speed, accepting possible data loss
Needs a strict majority	**3 nodes tolerate 1 failure, 5 tolerate 2.** During a network split, only the **majority side** makes progress

Limitation	Detail
Fixed membership	Most algorithms assume a **fixed set of voters**. **Dynamic membership** extensions exist but are **less well understood**
Timeout-based failure detection	With variable delays (especially across regions), nodes **falsely suspect the leader**. **Frequent elections** wreck performance
Sensitive to bad links	**Raft edge case:** one consistently unreliable link can make **leadership bounce** between two nodes forever, so there's no progress. This is still an open research problem

9. Conclusion

Consensus gets nodes to **agree irrevocably** despite failures. It underlies **leader election, atomic commit, locks, uniqueness and total ordering**. Formally it needs **agreement, integrity, validity** (safety) and **termination** (liveness), and termination requires a **working majority**. FLP says it's impossible without timeouts, but **timeouts and randomness make it practical**. **Lamport timestamps** give a causal total order but can't decide things **in the moment**. **Total order broadcast** can, and it's **equivalent** to consensus and to linearizable compare-and-set. Real algorithms (**Paxos, Raft, Zab, VSR**) implement total order broadcast with a **leader per epoch** (ballot, view or term). Each decision needs a **quorum that overlaps the election quorum**, so a stale leader can't win. Unlike **2PC**, they **elect** leaders and need only a **majority**, so they don't block. The costs are **synchronous voting, majority requirements (3 nodes for 1 failure), fixed membership, timeout sensitivity and edge cases with flaky links**. **Use ZooKeeper or etcd rather than building your own.**

Interview one-liners

- **Consensus:** "Nodes agree on a value such that no two decide differently, no one changes their mind, the value was proposed, and every live node eventually decides. It needs a majority alive."
- **FLP:** "Deterministic consensus is impossible in a fully asynchronous system with crashes. Timeouts make it practical."
- **Total order broadcast:** "Every node delivers the same messages in the same order. It's equivalent to consensus and to linearizable compare-and-set."
- **Epochs:** "A unique leader per term or ballot. Decisions need a quorum that overlaps the election quorum, so a stale leader can't commit."
- **vs 2PC:** "Consensus elects its leader and needs a majority. 2PC has a fixed coordinator, needs everyone, and blocks."

Self-check questions

1. Give four situations that need consensus.
2. Causality vs total order: why is causal consistency attractive?
3. How do Lamport timestamps work? Why can't they enforce unique usernames?
4. Define total order broadcast. How do you build linearizable compare-and-set from it?
5. State the four properties of consensus. Which is liveness?
6. Why does 2PC fail the termination property?
7. What does FLP say, and why can we still build consensus systems?
8. How do epoch numbers and overlapping quorums solve "to elect a leader you need a leader"?
9. How many nodes do you need to tolerate 2 failures?
10. List five limitations of consensus.

06 · Raft — Leader Election — Revision Notes

> 📘 **Source:** DDIA (Chapter 9) only **names** Raft as a consensus algorithm (used by etcd), with its **term numbers** and an edge case with unreliable links. These notes follow the **Raft paper**: *"In Search of an Understandable Consensus Algorithm"*, Diego Ongaro & John Ousterhout, USENIX ATC 2014, and Ongaro's PhD thesis. Folder guide: README.md · Previous: 05-Distributed-Consensus.md · Next: 07-Raft-Log-Replication.md

1. What Raft is

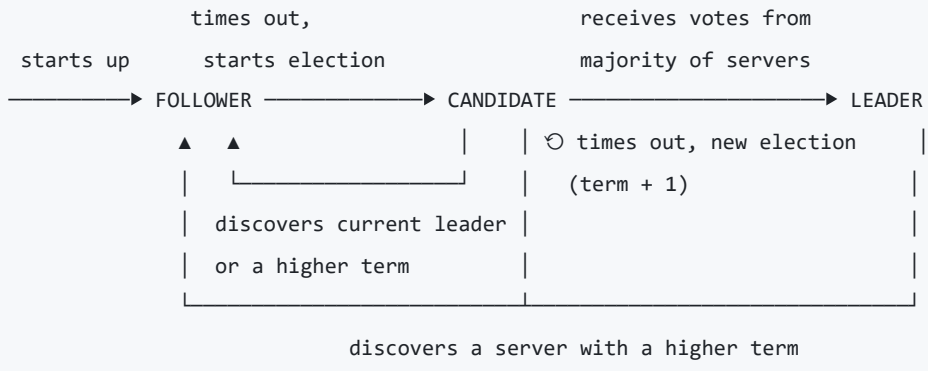
- A **consensus algorithm** for managing a **replicated log**, which is **total order broadcast** (see 05).
- It was designed to be **understandable**, because Paxos is famously hard. It splits the problem into three parts:
  1. **Leader election**: this file.
  2. **Log replication** (writes): 07.
  3. **Safety**: covered in both.
- **Strong leader:** log entries flow **only from leader to followers**.
- A cluster is usually **5 servers**, which **tolerates 2 failures** (it needs a majority of 3).

Used by: etcd (Kubernetes), Consul, CockroachDB, TiKV, and Kafka KRaft mode.

2. Server states

Every server is in **exactly one** state:

State	Role
Follower	**Passive**: only responds to the leader and candidates. Everyone **starts here**
Candidate	Trying to **become leader** in an election
Leader	Handles **all client requests** and **replicates the log**. At most **one per term**



3. Terms: Raft's epoch numbers

- Time is divided into **terms** numbered **1, 2, 3, ...**. They're the **epoch numbers** from DDIA.
- Each term **starts with an election**. If a candidate wins, it **leads for the rest of the term**.

- A term can end with **no leader** (a **split vote**), and then a new term starts right away.
- ⭐ **At most one leader per term.**

```
term:   |   1   |   2   | 3 |   4   ...
        |elect| normal  |elect|normal|elect(split vote, no leader)|elect| normal ...
```

Rules every server follows:

- It stores `currentTerm` and includes it in **every message**.
- It sees a **higher term** → updates `currentTerm` and **immediately becomes a follower**. A stale leader steps down.
- It gets a request with a **stale (lower) term** → **rejects** it.

➡️ Terms work as a **logical clock** that detects **stale leaders**.

4. Heartbeats and election timeouts

- The leader sends periodic **heartbeats**: **empty** `AppendEntries` **RPCs**.
- If a follower hears **nothing** for its **election timeout**, it assumes the leader is dead and **starts an election**.
- ⭐ **Election timeouts are randomized** (the paper suggests **150–300 ms**). Usually **one server times out first**, wins, and sends heartbeats before anyone else times out. This **avoids repeated split votes**.

5. How an election works

When a follower's timeout fires:

1. **Increment** `currentTerm`**.**
2. Become a **candidate**.
3. **Vote for itself.**
4. Send `RequestVote` **RPCs** to every other server **in parallel**.

`RequestVote` RPC

Arguments	Meaning
`term`	Candidate's term
`candidateId`	Who's asking
`lastLogIndex`	Index of the candidate's last log entry
`lastLogTerm`	Term of the candidate's last log entry

Reply	Meaning
`term`	Voter's current term (so the candidate can update itself)
`voteGranted`	true or false

Three possible outcomes

Outcome	What happens
1. It wins (votes from a **majority**)	Becomes **leader** and immediately sends **heartbeats** to establish authority and stop new elections
2. Another server wins	It receives an `AppendEntries` from a leader whose **term ≥ its own** and **steps back to follower**
3. No one wins (split vote)	It **times out again** (randomized), **increments the term**, and starts a **new election**

```
5 servers, S1 times out first (term 4 → 5)
S1: votes for itself — RequestVote(term 5) →► S2 ✅  S3 ✅  S4 ❌(down)  S5 ✅
S1 has 4/5 votes ≥ majority (3) → LEADER for term 5 → sends heartbeats
```

6. Voting rules (what makes it safe)

1. ⭐ **One vote per term:** each server votes for **at most one candidate per term**, **first come, first served**. Its vote (`votedFor`) is **saved to disk** so a restart can't make it vote twice. Combined with "majority needed", this means **two leaders can never win the same term** (**Election Safety**).
2. ⭐ **Election restriction (up-to-date log):** a server **refuses** to vote for a candidate whose log is **less up to date** than its own:
   - Compare the **term of the last entry**: the **higher** term is more up to date.
   - If the last terms are **equal**, the **longer** log is more up to date.

   Why: a candidate needs a majority, and every **committed** entry is on a majority. The two majorities **overlap**, so the winner's log must contain **every committed entry**. This is **Leader Completeness**: a new leader never lacks committed data.

```
Voter's log:      [t1][t1][t2][t3]          last term = 3
Candidate A log:  [t1][t1][t2]              last term = 2 → REJECT (less up to date)
Candidate B log:  [t1][t1][t2][t3][t3]      last term = 3, longer → GRANT
```

7. What each server saves to disk

Persistent (on disk, before replying)	Volatile
`currentTerm` : latest term seen	`commitIndex` : highest entry known to be committed
`votedFor` : candidate voted for this term	`lastApplied` : highest entry applied to the state machine
`log[]` : the entries (command + term)	Leader only: `nextIndex[]` , `matchIndex[]` (see 07)

8. Timing requirement

> **broadcastTime ≪ electionTimeout ≪ MTBF**

Quantity	Typical value
broadcastTime (time to send RPCs to every server and get replies)	0.5–20 ms
electionTimeout	10–500 ms
MTBF (mean time between failures of one server)	Months

If elections take a big share of the time, the system spends its time electing instead of working. DDIA warns about exactly this with **variable network delays**.

9. Known problems and fixes 📘

Problem	Fix (Ongaro's thesis and real implementations)
A partitioned node keeps bumping its term. It can't win, but when it rejoins, its **high term forces the healthy leader to step down**	**Pre-vote:** before raising its term, a candidate checks whether it *could* win. If not, it doesn't disrupt anyone
Leadership bouncing on one flaky link (the DDIA edge case)	**Pre-vote + CheckQuorum:** a leader steps down if it can't reach a majority, and followers ignore vote requests while they still hear a live leader
Removed servers disrupting elections	Ignore `RequestVote` while within the minimum election timeout of hearing from a leader

10. Conclusion

Raft elects a **single strong leader per term**. Servers are **followers, candidates or leaders**. **Terms** are increasing epoch numbers carried on every message, and a server that sees a higher term **steps down**, which neutralizes stale leaders. Leaders send **heartbeats**. A follower that hears nothing for its **randomized election timeout** (about 150–300 ms) **increments its term, votes for itself and sends** `RequestVote`. A **majority** makes it leader. Hearing from a legitimate leader makes it a follower again, and a **split vote** triggers a new term. Safety comes from **one persisted vote per term** (so at most one leader per term) and the **up-to-date log restriction** (compare the last term, then length), which guarantees **the new leader has every committed entry**. Elections must be rare relative to work (**broadcastTime** ≪ **electionTimeout** ≪ **MTBF**), and **pre-vote and CheckQuorum** fix disruptive or flaky-link cases.

Interview one-liners

- **Election:** "A follower times out after a randomized interval, increments its term, votes for itself and requests votes. A majority makes it leader for that term."
- **Why randomized timeouts:** "So usually only one server times out first and wins, which avoids repeated split votes."
- **Safety:** "One durable vote per term means at most one leader per term. Voters only accept candidates whose log is at least as up to date, so the leader has every committed entry."
- **Terms:** "A logical clock. A higher term always wins, so stale leaders step down immediately."

Self-check questions

1. What are the three server states and their transitions?
2. What is a term, and how does it relate to DDIA's epoch numbers?
3. What happens when a server sees a higher term? A lower one?

4. Why are election timeouts randomized?
5. Walk through an election: what does a candidate do, and what are the three outcomes?
6. What arguments does `RequestVote` carry, and why `lastLogIndex` and `lastLogTerm`?
7. Explain the up-to-date log check with an example.
8. Why must `votedFor` and `currentTerm` be saved to disk?
9. State the timing requirement and why it matters.
10. What problem does pre-vote solve?

07 · Raft — Log Replication (How Writes Work) — Revision Notes

> 📘 **Source:** the **Raft paper** (Ongaro & Ousterhout, 2014) and Ongaro's thesis. DDIA covers the ideas behind it: total order broadcast, quorums, epochs and linearizable reads (05, 03). Folder guide: README.md · Previous: 06-Raft-Leader-Election.md · Next: 08-ZooKeeper.md

1. The idea: a replicated state machine

```
client ──"SET x=5"──▶ LEADER
                        │ 1. append to own log
                        │ 2. AppendEntries to followers (parallel)
                        ▼
  Leader log:   [1:t1 SET a=1][2:t1 SET b=2][3:t2 SET x=5]
  Follower 1:   [1:t1 SET a=1][2:t1 SET b=2][3:t2 SET x=5]  ✅ ack
  Follower 2:   [1:t1 SET a=1][2:t1 SET b=2][3:t2 SET x=5]  ✅ ack
  Follower 3:   [1:t1 SET a=1][2:t1 SET b=2]                (slow)
  Follower 4:   (down)
                        │ 3. majority (3 of 5) has entry 3 → COMMITTED
                        │ 4. apply to state machine → x = 5
client ◀──────── "OK" ──────┘ 5. reply to client
```

- Each **log entry** = **(index, term, command)**.
- Every server applies the **same commands in the same order**, so every **state machine is identical**. That's **total order broadcast**.

2. The write path step by step

1. A **client sends a command to the leader**. Followers **redirect** clients to the leader.
2. The leader **appends** the entry to **its own log**.
3. It sends `AppendEntries` RPCs to all followers **in parallel**.
4. Once the entry is stored on a **majority** of servers (leader included), it is **committed**.
5. The leader **applies** it to its state machine and **returns the result to the client**.
6. The leader includes its `commitIndex` in later `AppendEntries` (and heartbeats), so followers learn what's committed and **apply it too**.

- Slow, crashed or lossy followers: the leader **retries** `AppendEntries` **indefinitely** until every follower has every entry.
- The **write latency** is **one round trip to a majority**. That's the "synchronous replication" cost DDIA mentions.

`AppendEntries` RPC

Arguments	Meaning
`term`	Leader's term
`leaderId`	So followers can redirect clients
`prevLogIndex`	Index of the entry **just before** the new ones
`prevLogTerm`	Term of that entry
`entries[]`	New entries (**empty = heartbeat**)
`leaderCommit`	Leader's `commitIndex`

Reply	Meaning
`term`	For the leader to update itself (and step down if it's stale)
`success`	true if the follower had an entry matching `prevLogIndex` and `prevLogTerm`

3. Keeping logs consistent: the Log Matching Property

> ⭐ **If two logs have an entry with the same index and term, then (1) the entries hold the same command, and (2) the logs are identical in all earlier entries.**

1. A leader creates **at most one entry per index in a term**, and entries **never move**.
2. **The consistency check:**
   - Every `AppendEntries` carries `prevLogIndex` and `prevLogTerm`.
   - A follower **rejects** it if it has no entry at `prevLogIndex` with term `prevLogTerm`.
   - So each accepted append proves the logs **match up to that point**, by induction.

4. Repairing a follower's log

After crashes and leader changes, a follower may be **missing** entries or have **extra, uncommitted** entries from old leaders.

The leader keeps a `nextIndex[f]` for each follower: the next entry to send. It starts just after the leader's last entry.

```
Leader (term 6):  idx: 1  2  3  4  5  6  7
                  trm: 1  1  1  4  4  5  6
Follower:         idx: 1  2  3  4  5  6
                  trm: 1  1  1  2  2  2      ← stale entries from an old term-2 leader

AppendEntries(prevLogIndex=7, prevLogTerm=6) → follower has no entry 7 → REJECT
leader: nextIndex-- → try prevLogIndex=6 (term 5) → follower has term 2 there → REJECT
leader: nextIndex-- → try prevLogIndex=5 (term 4) → REJECT
leader: nextIndex-- → try prevLogIndex=4 (term 4) → REJECT
leader: nextIndex-- → try prevLogIndex=3 (term 1) → MATCH ✅
follower DELETES its conflicting entries 4–6 and appends the leader's 4–7
```

- ⭐ **The leader never overwrites or deletes its own log** (**Leader Append-Only**). It forces followers to **match it**.
- Only **uncommitted** entries are ever thrown away. Committed entries are safe (see §5).
- *Optimization:* the follower returns the **conflicting term and its first index**, so the leader can skip a whole term per round trip.

5. When is an entry committed? (the subtle rule)

- An entry from the **current term** is **committed once it's on a majority**.
- ⚠️ A leader **does not commit entries from earlier terms by counting replicas**. An old entry on a majority could still be **overwritten** by a future leader that never saw it (the famous "Figure 8" scenario in the paper).
- Instead, older entries are committed **indirectly**: when an entry from the **current term** is committed, **every entry before it** is committed too (Log Matching).
- That's why a new leader immediately appends a **no-op entry** at the start of its term: committing it also commits everything earlier.

6. Raft's safety guarantees

Property	Guarantee	Comes from
Election Safety	At most **one leader per term**	One vote per term + majority
Leader Append-Only	A leader **never overwrites or deletes** its log entries	Design rule
Log Matching	Same index and term → same entry **and** identical prefix	Consistency check
Leader Completeness	A committed entry is in the log of **every future leader**	Election restriction (06 §6) + commit rule
State Machine Safety	No two servers apply **different commands at the same index**	All of the above

7. Reads and clients

Linearizable reads (DDIA: reads can be stale)

A leader might have been **replaced without knowing it**, which would give a **stale read**. The fixes:

- **Read through the log:** append the read as an entry and answer it when committed. Simple but slow.
- **ReadIndex:** the leader records its `commitIndex`, **confirms it's still leader** with a round of heartbeats to a majority, waits until it has applied up to that index, then answers. (DDIA's etcd "quorum read" works this way, and ZooKeeper's `sync()` is similar.)
- **Lease-based reads:** the leader assumes it's still leader for a while after heartbeats. Faster, but **relies on bounded clock drift**.
- A new leader must first commit its **no-op entry**, so it knows what's committed.

Exactly-once commands

If a leader crashes after committing but before replying, the **client retries**, and the command could run **twice**. The fix is for clients to tag commands with **unique serial numbers**, and the state machine **ignores duplicates**.

8. Practical extensions

Topic	Idea
Log compaction / snapshots	The log grows forever, so periodically **snapshot the state machine** and **discard the log prefix**. A far-behind follower gets an `InstallSnapshot` RPC
Membership changes	Add or remove servers safely with **joint consensus** (old and new configurations overlap), or by changing **one server at a time** (Ongaro's thesis). DDIA notes dynamic membership is less well understood
Batching and pipelining	Send many entries per RPC and don't wait for each round trip, which gives higher throughput

9. Raft vs single-leader database replication

	Single-leader DB (Chapter 5)	Raft
Leader election	Manual, or ad hoc automatic failover	**Built in**, majority vote, terms
Split brain	Possible (needs fencing / STONITH)	**Prevented** (one leader per term, stale terms rejected)
Commit	Often **async**, so failover can **lose committed writes**	Committed = **on a majority**, so **never lost** while a majority survives
Linearizable	Maybe	✅ Yes (with ReadIndex or log reads)

10. Conclusion

In **Raft log replication**, every write goes to the **leader**, which **appends** an entry (index, term, command) and sends `AppendEntries` to followers in parallel. Once the entry is on a **majority**, it's **committed**, applied to the state machine and acknowledged to the client, and followers learn it via `leaderCommit`. The **consistency check** (`prevLogIndex` and `prevLogTerm`) enforces the **Log Matching Property**. When a follower diverges, the leader **backs up** `nextIndex` until the logs match and then **overwrites the follower's uncommitted extras**, never its own log. A leader only commits **current-term** entries by counting, so older entries commit **indirectly**, which is why it starts each term with a **no-op**. Together with the election restriction, this gives **Election Safety, Log Matching, Leader Completeness and State Machine Safety**: every server applies the same commands in the same order, and **committed writes are never lost**. **Linearizable reads** need a log read, **ReadIndex** or leases. **Client serial numbers** give exactly-once, and **snapshots and membership changes** keep it practical. This is why etcd, Consul, CockroachDB and Kafka KRaft use it.

Interview one-liners

- **Raft write:** "The client sends to the leader, which appends to its log and replicates with AppendEntries. Once a majority stores it, the entry is committed and applied, and the leader replies."
- **Consistency check:** "AppendEntries includes the previous entry's index and term. A follower rejects a mismatch, so logs that agree at one point agree on everything before it."
- **Repair:** "The leader decrements nextIndex until the logs match, then the follower drops its conflicting uncommitted entries and copies the leader's."
- **Subtle rule:** "A leader only commits entries from its own term by counting replicas. Earlier ones commit indirectly, hence the no-op at the start of each term."

- **Reads:** "To be linearizable, the leader must confirm it's still leader (ReadIndex) or put the read through the log."

Self-check questions

1. Walk through a Raft write from client to reply.
2. What's in a log entry? What's in an `AppendEntries` RPC?
3. When is an entry committed? How do followers find out?
4. State the Log Matching Property and how the consistency check enforces it.
5. Walk through repairing a follower with stale entries from an old term.
6. Why does the leader never delete its own entries?
7. Why can't a leader commit an old-term entry just by counting replicas? What's the no-op for?
8. List Raft's five safety properties.
9. How do you make reads linearizable in Raft?
10. How do client serial numbers give exactly-once semantics?
11. What are snapshots and joint consensus for?
12. How does Raft improve on ordinary single-leader replication?

08 · ZooKeeper (and Coordination Services) — Revision Notes

> Source: *Designing Data-Intensive Applications*, **Chapter 9**, "Membership and Coordination Services" (pp. 370–373), plus Chapter 6 "Request Routing" and Chapter 9 on locks and fencing tokens. Sections marked 📘 **Beyond the book** add ZooKeeper's data model and recipes (from the ZooKeeper docs and paper) that interviews expect. Folder guide: README.md · Previous: 07-Raft-Log-Replication.md

1. What ZooKeeper is

> **ZooKeeper** is a **coordination service**: a small, highly reliable, **consensus-replicated key-value store** that distributed systems use to **agree on things** such as who the leader is, who holds a lock, which node owns which partition, and who is alive.

- It's described as a "distributed key-value store" or a "coordination and configuration service". Its API looks like a database: read and write keys, list keys.
- **Why use consensus at all?** Because the data it holds must be **agreed on by everyone**.
- Modelled after **Google's Chubby** lock service.
- You rarely use it **directly**. You rely on it **through** other systems: **HBase, Hadoop YARN, OpenStack Nova, Kafka** (historically), SolrCloud.
- **Similar systems: etcd** (uses **Raft**, and is the backbone of **Kubernetes**) and **Consul**.

What it's NOT for:

- ❌ Not a general-purpose database. It holds **small data that fits in memory** (still written to disk for durability).
- ❌ Not for **fast-changing runtime state**. Its data changes over **minutes or hours** ("10.1.1.23 is leader for partition 7"), not thousands of times a second. For replicating application state, use tools like **Apache BookKeeper**.

2. How it's built

- A small, fixed **ensemble**, usually **3 or 5 servers**, so a majority survives 1 or 2 failures.
- Data is replicated to all servers with **fault-tolerant total order broadcast** using the **Zab** protocol (ZooKeeper Atomic Broadcast).
- 📘 **One leader handles all writes** and followers replicate them. **Reads are served locally by any server**, so they're fast but **may be stale**.
- 📘 **Observers** are non-voting servers that receive the log but don't vote. They add **read capacity** without slowing votes. This is DDIA's "**read-only caching replicas**".
- ⭐ **Outsourcing coordination:** the app may grow to **thousands of nodes**, but the **majority votes run only among 3 to 5 ZooKeeper servers**. Consensus, operation ordering and failure detection are **delegated** to an external service.

3. The four key features (DDIA)

Feature	What it gives you	Needs consensus?
Linearizable atomic operations	**Compare-and-set**, so you can build a **lock**: if several nodes try at once, **exactly one succeeds**, even during failures. Locks are usually **leases** (they expire if the client dies)	✅ **Yes**, the only one that truly needs it
Total ordering of operations	Every operation gets a monotonically increasing **transaction ID (`zxid`)** and version (`cversion`). These are **fencing tokens** that stop a paused old lock holder from corrupting data	—
Failure detection	Clients keep a **session** with **heartbeats**. If heartbeats stop for longer than the **session timeout**, the session is dead and its **ephemeral nodes** (and locks) are **released automatically**	—
Change notifications (watches)	Clients **watch** keys and get **notified** when they change, for example a node joins, or a node dies and its ephemeral node disappears. **No polling** needed	—

> The **combination** of these features is what makes ZooKeeper useful for coordination.

Fencing tokens: why total ordering matters

```
Client 1 gets lock (token 33) → long GC pause 💤 … lease expires
Client 2 gets lock (token 34) → writes to storage with token 34 ✅
Client 1 wakes up, still thinks it holds the lock → writes with token 33
Storage: "33 < 34, reject" ✅  (the zxid from ZooKeeper is the token)
```

> ⚠️ ZooKeeper's **writes are linearizable**, but **reads may be stale** by default. Call `sync()` before a read when you need it to be linearizable.

4. Data model 📘 Beyond the book

A **hierarchical namespace like a filesystem**. Each node is a **znode** that holds small data (≤ 1 MB, typically much less) and can have children.

```
/
├── app1
│   ├── config           (persistent: "db_url=...")
│   ├── leader           (ephemeral: "node-3")
│   └── workers
│       ├── worker-0000000001   (ephemeral + sequential)
│       └── worker-0000000002
└── locks
    └── resource-x
        ├── lock-0000000007     (ephemeral + sequential)
        └── lock-0000000008
```

znode type	Behaviour
Persistent	Stays until explicitly deleted
Ephemeral	**Deleted automatically when the creating session dies**, which is the basis of failure detection
Sequential	ZooKeeper appends a **monotonically increasing counter** to the name (`-0000000007`)
Ephemeral + sequential	Both. The building block for **locks and leader election**

- **Watches** are **one-time triggers**: after firing, you must **re-register** them.
- **Guarantees: linearizable writes**, **FIFO order per client**, and every client sees updates in the **same order** (sequential consistency).

5. What ZooKeeper is used for

(a) Leader election and allocating work (DDIA)

- Several instances of a service, **exactly one** of which must be leader (single-leader databases, **job schedulers**), and **someone else takes over** if it fails.
- **Assigning partitions to nodes** and **rebalancing** when nodes join or fail (see 01 §8).
- Done with **atomic operations, ephemeral nodes and notifications**. Libraries like **Apache Curator** provide recipes. It's still hard, but **far better than writing your own consensus**.

📘 **The leader election recipe:**

```
1. Each candidate creates an EPHEMERAL + SEQUENTIAL znode: /election/n_0000000005
2. List children. The LOWEST sequence number is the leader.
3. Every non-leader WATCHES only the znode just before it (not the leader),
   which avoids the "herd effect" of everyone waking up at once
4. Leader dies → its session expires → its ephemeral znode vanishes
   → the next node gets notified → it re-checks and becomes leader
```

A **distributed lock** works the same way: the lowest sequence number holds the lock.

(b) Service discovery (DDIA)

- Services **register their IP and port** on startup, and others **look them up**.
- Used with ZooKeeper, **etcd** and **Consul**.
- ⚠️ Service discovery **doesn't strictly need consensus**. **DNS** works well and is not linearizable, and staleness is fine. But **leader election does**, so it makes sense to reuse the consensus system to tell services **who the leader is**. Read-only replicas (observers) serve those non-linearizable reads.

(c) Membership services (DDIA)

- Decide which nodes are **currently alive members** of the cluster. This research dates back to the 1980s (e.g. **air traffic control**).
- You **can't reliably detect failure** over a network, but **failure detection plus consensus** lets nodes **agree** on who's alive.
- A node may be **wrongly declared dead**, but **agreement** is what matters. For example, "leader = the lowest-numbered member" only works if everyone agrees on the member list.

(d) Configuration and metadata 📘

- Small shared config (feature flags, connection strings) that everyone **watches** for changes.
- **Partition maps** for routing tiers: HBase, SolrCloud, Kafka (historically), and LinkedIn Espresso via Helix.

6. ZooKeeper vs etcd vs Consul 📘 Beyond the book

	ZooKeeper	**etcd**	**Consul**
Consensus	**Zab**	**Raft**	**Raft**
Data model	Hierarchical znodes	Flat key-value (with prefixes)	Key-value + service catalog
Failure detection	Sessions + ephemeral nodes	**Leases** (TTL)	Health checks + sessions
Notifications	One-time watches	Streaming watches	Blocking queries / watches
Known for	Hadoop ecosystem, HBase, (older) Kafka	**Kubernetes**	Service discovery and mesh

> 📘 **Kafka moved away from ZooKeeper: KRaft** mode (Kafka's own **Raft**-based metadata quorum) replaced it. ZooKeeper support was **removed in Kafka 4.0**. It's a good example of a system moving consensus **in-house**.

7. Conclusion

ZooKeeper (like etcd and Consul) is a **coordination service**: a small, in-memory, **consensus-replicated** store (Zab, on a fixed **3 or 5 server** ensemble) that distributed systems use to **agree on shared facts**. It isn't a general database. It provides **linearizable atomic operations** (compare-and-set locks and leases, the only feature that truly needs consensus), **total ordering** (zxid as **fencing tokens**), **failure detection** (sessions and heartbeats that delete **ephemeral nodes**), and **change notifications** (watches). Combined, these power **leader election and work allocation** (ephemeral sequential znodes: lowest wins, each watches its predecessor), **partition assignment and request routing**, **service discovery** (which doesn't strictly need consensus, but can reuse the leader info) and **membership services**. Writes are linearizable, but reads may be stale unless you call `sync()`. It lets thousands of app nodes **outsource consensus** to a few servers instead of implementing it themselves, though systems like Kafka (KRaft) now embed Raft directly.

Interview one-liners

- **What is ZooKeeper:** "A consensus-backed coordination service (Zab, 3 to 5 servers) for small, slowly changing shared state: leader election, locks, config, membership and partition maps."
- **Key primitives:** "Linearizable compare-and-set, zxid ordering for fencing tokens, sessions with ephemeral nodes for failure detection, and watches for notifications."
- **Leader election recipe:** "Create an ephemeral sequential znode. The lowest number is leader. Everyone else watches their predecessor, and when the leader dies its znode disappears and the next one takes over."
- **Caveat:** "Writes are linearizable, but reads can be stale unless you call sync(). Don't use it as a general database."

Self-check questions

1. What is ZooKeeper for, and what is it NOT for?
2. Why does ZooKeeper use consensus? Which protocol?

3. List DDIA's four key features. Which one actually requires consensus?
4. How do zxids work as fencing tokens? Walk through the GC-pause example.
5. What are ephemeral and sequential znodes?
6. Describe the leader election recipe and how it avoids the herd effect.
7. How does ZooKeeper help with partition assignment and request routing?
8. Does service discovery need consensus? Why might you still use ZooKeeper for it?
9. What is a membership service, and why is agreement more important than accuracy?
10. Are ZooKeeper reads linearizable? How do you make them so?
11. Compare ZooKeeper, etcd and Consul. Why did Kafka move to KRaft?

Distributed Systems — Folder Guide & Summary of Everything

> Sources: *Designing Data-Intensive Applications* (Martin Kleppmann), **Chapter 6 (Partitioning)** and **Chapter 9 (Consistency and Consensus)**, plus the **Raft paper** (Ongaro & Ousterhout, 2014) for the Raft files. 📄 **Everything in one PDF:** Distributed-Systems-Complete-Notes.pdf Sections marked 📘 go beyond the book.

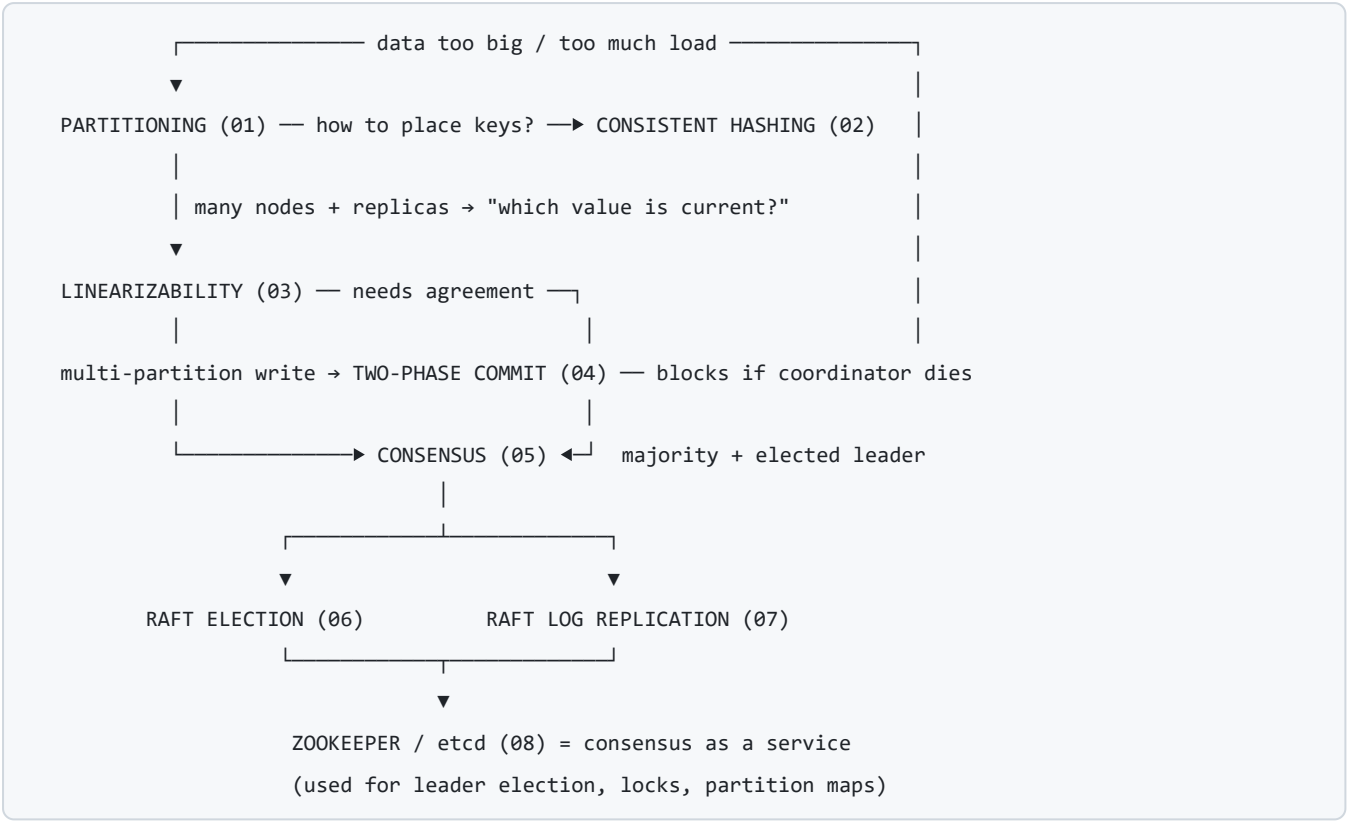
📚 Reading order

#	File	What it covers	Source
01	Partitioning	Key-range vs hash, hot spots, secondary indexes, rebalancing, request routing	DDIA Ch. 6
02	Consistent Hashing	Hash ring, virtual nodes, replication on the ring, mod-N problem	DDIA Ch. 6 + 📘
03	Linearizability	Recency guarantee, vs serializability, use cases, CAP, cost	DDIA Ch. 9
04	Two-Phase Commit	Atomic commit, 2PC, coordinator failure, XA, in-doubt locks	DDIA Ch. 9
05	Distributed Consensus	Ordering, total order broadcast, consensus properties, FLP, epochs and quorums	DDIA Ch. 9
06	Raft: Leader Election	States, terms, randomized timeouts, RequestVote, election restriction	📘 Raft paper
07	Raft: Log Replication (Writes)	AppendEntries, commit rule, log repair, safety, linearizable reads	📘 Raft paper
08	ZooKeeper	Coordination services, znodes, fencing tokens, recipes, etcd and Consul	DDIA Ch. 9 + 📘

Why this order:

1. **Partitioning** and **consistent hashing** spread the data out.
2. Spreading data raises the question of **consistency** (linearizability).
3. Writing across partitions needs **atomic commit** (2PC).
4. 2PC's weaknesses lead to **consensus**.
5. **Raft** shows how consensus actually works.
6. **ZooKeeper** packages consensus as a service.

1. The big picture



2. Partitioning in one table

Topic	Key point
Key-range partitioning	Sorted, so **range scans** are cheap, but **hot spots** appear (timestamps)
Hash partitioning	**Even load**, but **no range queries**. Cassandra compound keys are the compromise
Hot key	The application splits it (random suffix), and reads must merge
Local secondary index	1-partition writes, **scatter/gather reads**
Global secondary index	1-partition reads, multi-partition, **async** writes
Rebalancing	❌ `mod N`. ✅ Fixed partitions, dynamic split and merge, or partitions per node
Routing	Any node / routing tier / smart client, with **ZooKeeper** or **gossip** holding the map

3. Consistent hashing in one line

Nodes and keys sit on a **ring**, and a key goes to the **next node clockwise**. Adding or removing a node moves only **about 1/N of the keys**. **Virtual nodes** even out the load.

4. Linearizability and CAP in one line

Behave like **one copy with atomic operations**: after a write, **everyone** sees it. It's needed for **locks, leader election and uniqueness**. During a partition, choose **consistency or availability**, and it's **slow even without faults**.

5. 2PC vs consensus

	2PC	**Consensus (Raft, Zab, Paxos)**
Goal	Atomic commit across nodes	Agree on values or an ordered log
Leader	Fixed **coordinator**	**Elected** per term or epoch
Votes	**All** must say yes	**Majority**
On leader failure	**Blocks** (in doubt, locks held)	New election, continues
Termination	❌	✅ (majority alive)

6. Consensus essentials

- **Properties:** uniform agreement, integrity, validity (**safety**) + termination (**liveness**, needs a majority).
- **FLP:** impossible without timeouts. **Timeouts and randomness** make it practical.
- **Equivalences:** linearizable compare-and-set ⇔ total order broadcast ⇔ consensus.
- **Epochs + overlapping quorums:** a unique leader per term, and stale leaders can't commit.
- **Costs:** synchronous voting, **2f + 1 nodes to tolerate f failures**, fixed membership, timeout sensitivity.

7. Raft on one page

	Leader election	**Log replication (writes)**
Trigger	Follower hears no heartbeat within a **randomized timeout** (about 150–300 ms)	Client sends a command to the **leader**
Action	Term + 1 → candidate → vote for self → **RequestVote**	Append to own log → **AppendEntries** to followers
Success	Votes from a **majority** → leader → heartbeats	Stored on a **majority** → **committed** → apply → reply
Safety rule	**1 vote per term** + vote only for a candidate with an **up-to-date log**	**prevLogIndex/prevLogTerm** check. Commit only **current-term** entries by counting (no-op at start of term)
Guarantees	Election Safety	Log Matching, Leader Completeness, State Machine Safety

8. ZooKeeper on one page

- **What:** a consensus-replicated (**Zab**) coordination store on **3 or 5 servers** for **small, slowly changing** data.
- **Primitives:**
  - **linearizable compare-and-set** (locks and leases)
  - **zxid ordering** (fencing tokens)
  - **sessions + ephemeral znodes** (failure detection)
  - **watches** (notifications)
- **Recipes:** leader election and locks (lowest **ephemeral sequential** znode wins, others watch their predecessor), partition maps, service discovery, membership, config.
- **Caveats:** reads may be stale (use `sync()`). Not a general database. Alternatives: **etcd** (Raft, Kubernetes) and **Consul**. Kafka replaced it with **KRaft**.

9. 🎯 Interview cheat sheet

Question	Short answer
What is partitioning / sharding?	Splitting data across nodes so each holds a subset, for scale. Usually combined with replication
Range vs hash partitioning?	Range: ordered scans, hot spots. Hash: even load, no range queries
How do you fix a hot key?	Split it with random suffixes at the application level, and merge on read
Local vs global secondary index?	Local: cheap writes, scatter/gather reads. Global: cheap reads, costly async writes
Why not `hash mod N`?	Changing N moves almost every key
What is consistent hashing?	A ring of nodes and keys where a key goes to the next node clockwise. Adding a node moves only about 1/N of keys. Virtual nodes even out the load
What is linearizability?	The illusion of a single copy. Once written, every read sees it
Linearizability vs serializability?	Recency on single objects vs transaction isolation. Both together is strict serializability
What does CAP really say?	During a network partition, choose linearizability or availability
What is 2PC?	Prepare (vote) and commit phases run by a coordinator, giving atomic commit across nodes
Main problem with 2PC?	Coordinator crash means participants are in doubt, block and hold locks
What is consensus?	Agreement + integrity + validity + termination. It needs a majority
What is FLP?	No deterministic consensus in an async system with crashes. Timeouts fix it in practice
How many nodes to tolerate f failures?	**2f + 1** (3 nodes for 1, 5 nodes for 2)
How does Raft elect a leader?	Randomized timeout → candidate (term + 1) → RequestVote → majority wins. One vote per term, up-to-date log check
How does a Raft write work?	Leader appends → AppendEntries → majority stored → committed → applied → reply
How are Raft logs kept consistent?	prevLogIndex/prevLogTerm check, then the leader backs up and overwrites the follower's uncommitted conflicting entries
What is ZooKeeper?	Consensus-backed coordination for leader election, locks, config, membership and partition maps
What is a fencing token?	An increasing number (zxid) sent with each write, so storage can reject a stale lock holder

10. Conclusion: everything in one paragraph

When data outgrows one machine we **partition** it, by **key range** (good scans, hot-spot risk) or by **hash** (even load, no scans), handle **secondary indexes** locally or globally, **rebalance** without `mod N` (fixed, dynamic or per-node partitions, or **consistent hashing** with virtual nodes), and **route** requests using a coordination service or gossip. With data spread over many replicas, **linearizability** gives the simple illusion of one copy, which locks, leader election and uniqueness need. It costs **availability during partitions (CAP)** and **latency all the time**. Writes spanning partitions need **atomic commit**. **Two-phase commit** gives it, but a **coordinator crash leaves participants in doubt** and holding locks. **Fault-tolerant consensus** fixes this: it **elects** a leader per **epoch or term** and needs only a **majority**, so it **terminates** despite failures (FLP is sidestepped with timeouts). It's equivalent to **total order broadcast** and **linearizable compare-and-set**. **Raft** makes consensus

understandable. **Randomized election timeouts**, **one vote per term** and an **up-to-date log check** elect a single safe leader. **AppendEntries** with a **consistency check**, **majority commit** and a **current-term commit rule** keep every log identical and committed writes durable. Finally, **ZooKeeper** (and etcd and Consul) packages consensus as a **service**: linearizable locks, zxid **fencing tokens**, **ephemeral nodes** for failure detection and **watches**. Real systems use it for leader election, partition maps, service discovery and membership instead of implementing consensus themselves.